RecoveryEval: Evaluating Continuation Inertia Under Injected Reasoning Errors in Language Models

Sham Satish Thakare
Independent Researcher
Pune, Maharashtra, India
Email: shamthakare@gmail.com

Abstract—Evaluating whether post-training enhances a language model's capacity to detect and recover from intermediate reasoning errors is critical for building reliable artificial intelligence systems. However, naive trajectory evaluations routinely conflate error recovery with baseline fluency, prompt length discrepancies, and truncation censoring. In this paper, we present RecoveryEval, a state-matched, provenance-aware evaluation framework that assesses reasoning trajectories under paired counterfactual prefix perturbations. We evaluate 400 genuine neural rollouts generated by two released checkpoints (Qwen2.5-Math-1.5B Base and Instruct) across 20 multi-step arithmetic problems under a length-balanced, truncation-free protocol (Protocol 2.0; 256-token budget).

Under symmetrically matched prefixes withholding the terminal answer, neither model exhibits spontaneous error recovery: across all 200 recovery continuations, zero spontaneous corrections are observed (0/200, 0.0%). Instead, models exhibit pronounced continuation inertia: in 98.5% of cases (197/200), models accept the injected error as an authoritative premise and continue derivations consistently from it, with the remaining 1.5% failing due to attention slips. Under an independent-rollout assumption ($n = 200$), the exact two-sided 95% Clopper-Pearson confidence interval is $[0.00\%, 1.83\%]$ (one-sided upper bound: 1.49%); accounting for intra-cluster correlation across the 20 problem templates yields an effective cluster-level sensitivity bound of $\leq$ 13.91% (two-sided interval: $[0.00\%, 16.84\%]$). On unperturbed control prefixes, both models achieve 100.0% structural accuracy, yielding an observed sample recovery contrast of $D = 0.0000$ (where sample variance collapses due to boundary saturation across the evaluated items). Trace forensics reveal that legacy regex parsers falsely penalize base models by 3% due to trailing review summaries, which structural equivalence resolves. Our findings demonstrate that small language models behave as faithful continuation engines rather than autonomous truth-checkers, highlight how naive evaluation designs manufacture severe confounding artifacts, and provide open-source cryptographic infrastructure for reasoning diagnostics.

Index Terms—Language Models, Mathematical Reasoning, Error Recovery, State-Matched Evaluation, Benchmark Governance, Continuation Inertia, Clopper-Pearson Bounds.

I. Introduction

Large language models (LLMs) specialized for mathematics have demonstrated remarkable gains on multi-step reasoning benchmarks [1], [2]. A central question in AI reliability is whether post-training procedures (such as instruction tuning, reinforcement learning from human feedback, or process supervision) instill the capacity to recognize and self-correct intermediate calculation mistakes [3], [4].

However, measuring recovery behavior is fraught with methodological pitfalls. Naive comparisons between unperturbed and error-injected trajectories frequently introduce severe confounding variables:

1) Control Answer Exposure: If control prefixes contain the ground-truth final answer while recovery prefixes withhold it, the evaluation measures pattern echoing rather than reasoning.
2) Truncation Censoring: Tight generation limits (e.g., 64 tokens) prematurely truncate longer reasoning traces, confounding verbosity with error recovery.
3) Evaluator Brittleness: Regular-expression parsers that extract the last numerical token falsely penalize models that conclude with conversational summary steps.
4) Pseudoreplication: Treating clustered rollouts as independent experimental units inflates degrees of freedom and fabricates statistical significance.

In preliminary experiments using an unbalanced 64-token protocol (Protocol 1.0), an apparent negative interaction was reported ($D \approx -0.21$), suggesting that instruction tuning degraded recovery resilience. However, Protocol 1.0 suffered simultaneously from 100% control answer leakage, 53.0% truncation censoring, and decorative matching formulas.

To resolve these ambiguities, we present RecoveryEval, an open-source evaluation framework built on Paired Counterfactual Prefix Perturbation Evaluation (PCPPE) and cryptographic event ledgers. Across 400 genuine neural rollouts evaluated under a length-balanced, 256-token budget (Protocol 2.0), we establish:

1) Non-Reproduction of Negative Interaction: Under Protocol 2.0, the historical negative interaction disappears ($D = 0.0000$ under structural verification; $D = -0.0300, p = 0.0828$ under heuristic parsing), proving that recovery estimates are acutely sensitive to protocol design.
2) Continuation Inertia: Small (1.5B) language mod-

els do not spontaneously self-correct from injected intermediate errors (0/200 recoveries). In 98.5% of cases, they treat corrupted premises as authoritative and faithfully propagate errors downstream.

3) Finite-Sample Uncertainty Bounds: We establish that observing zero recoveries corresponds to a one-sided 95% upper bound of 1.49% under independent rollout assumptions ($n = 200$), but cluster-level modeling reveals an effective sensitivity upper bound of 13.91% ($C = 20$).
4) Evaluator Sensitivity: Heuristic regex extractors artificially penalize base models by 3% due to trailing review summaries, whereas structural equivalence demonstrates 100% baseline accuracy for both checkpoints.

II. Related Work

Self-correction and error localization in language models have attracted intense scrutiny. Table I positions RecoveryEval relative to existing paradigms.

A. Intrinsic Self-Correction Limitations

Huang et al. [7] established that prompting models to self-correct (*"Review your answer and correct mistakes"*) without external feedback impairs performance because models alter correct answers more frequently than they repair incorrect ones. Stechly et al. [10] demonstrated similar self-verification failures in planning tasks. While these studies evaluated post-hoc multi-turn prompting, RecoveryEval investigates in-flight intermediate continuation: how autoregressive models behave when continuing from corrupted intermediate derivations without explicit reflection prompts.

B. External Feedback and Process Supervision

Shinn et al. [5] (Reflexion) and Gou et al. [6] (CRITIC) demonstrated that self-correction succeeds when models interact with external tools (code interpreters, calculators, search engines). Tyen et al. [8] (BIG-Bench Mistake) showed that while models fail to locate their own mistakes, they can correct them if the exact erroneous step is pointed out. Zheng et al. [9] (ProcessBench) and Wang et al. [11] (Math-Shepherd) confirmed the difficulty of automated process verification. RecoveryEval complements these studies by providing a clean baseline for unassisted continuation, demonstrating that without external tool feedback, continuation inertia completely dominates.

III. Research Questions and Causal Framing

We formulate three core empirical questions:

- RQ1 (Continuation vs. Recovery): When presented with an unprompted continuation prefix containing an erroneous intermediate calculation, does an autoregressive language model detect and correct the error, or does it propagate the corrupted premise downstream?
- RQ2 (Protocol Sensitivity): Does the apparent interaction between model alignment and error recovery reproduce when control answer leakage, truncation censoring, and parser artifacts are eliminated?
- RQ3 (Uncertainty Quantification): When zero recovery events are observed in clustered factorial data, what exact finite-sample confidence limits are statistically defensible?

A. Causal Identification Boundaries

We represent the data-generating process as a structural causal model:

- Prefix Intervention ($T \to Y$): Within each problem cluster P_i, the problem text and remaining token horizon are held constant. The only intervention is the substitution of the Step 1 product (s_C vs. s_R). The contrast in continuation accuracy between s_C and s_R is causally identified.
- Checkpoint Comparison ($M \to Y$): Comparing Base and Instruct models is observational. Base models expect raw completion strings, while Instruct models require ChatML formatting (<|im_start|>). We do not claim to isolate a pure causal post-training effect free from interface confounding.
- Scope of Construct: The experiment measures continuation inertia under externally supplied corruption, not an intrinsic inability to self-correct during uninterrupted, self-generated reasoning.

IV. Experimental Design: Protocol 2.0

A. Benchmark Construction

We evaluate 20 multi-step arithmetic word problems inspired by GSM8K ($C = 20$). Each problem involves an initial rate-multiplication step (Step 1) followed by a downstream multiplication step to determine total revenue (Step 2). Neither prefix reveals the final terminal answer.

B. Symmetrical Counterfactual Prefix Construction

For each problem P_i, two prefixes are constructed:

- Reference Control Prefix (s_C): Contains valid Step 1 arithmetic (e.g., $5 \times 4 = 20$).
- Recovery Perturbation Prefix (s_R): Introduces an arithmetic corruption into Step 1 (e.g., $5 \times 4 = 22$).

Both prefixes are truncated immediately prior to the Step 2 calculation, ensuring exact token and character balance ($\Delta_{\text{tokens}} = 0, \Delta_{\text{chars}} = 0$).

C. Factorial Matrix and Hardware Execution

The factorial design comprises:

20 problems×2 models×2 conditions×5 seeds = 400 rollouts

Evaluated checkpoints are Qwen/Qwen2.5-Math-1.5B (Base, Rev 4a83ca6e) and Qwen/Qwen2.5-Math-1.5B-Instruct (Instruct, Rev aafeb0fc) [2]. Rollouts were executed on an NVIDIA GeForce RTX 4090 GPU in native float16 precision using stochastic nucleus sampling ($T =$

TABLE I
Systematic Comparison of Reasoning and Self-Correction Evaluation Frameworks

Framework	Primary Target	Intervention Type	Prefix Symmetry	Core Empirical Finding
GSM8K [1]	End-to-end reasoning	None (Question only)	None	Scaling improves final outcome accuracy
PRM800K [3]	Process supervision	Step classification	Unmatched	Step-level oversight outperforms outcome oversight
Reflexion [5]	Multi-turn correction	Verbal reflection buffer	Asymmetric	External verbal feedback enables error recovery
CRITIC [6]	Tool-based correction	External interpreter/search	Interactive	Tool feedback enables calculation self-correction
Huang et al. [7]	Intrinsic self-correction	Prompted reflection	Post-hoc	Prompted reflection without feedback degrades accuracy
BIG-Bench Mistake [8]	Error localization	Explicit error location cue	Post-hoc traces	Models cannot find errors, but fix given location
ProcessBench [9]	Error step identification	Benchmark critic tasks	Varied	PRMs struggle to locate errors on competition math
RecoveryEval (Ours)	In-flight continuation inertia	Symmetrical step perturbation	Length-matched ($\Delta_{\text{tokens}} = 0$)	98.5% error propagation; 0% spontaneous recovery

$0.7, p = 0.9$) across 5 fixed seeds $(401, 402, 403, 404, 405)$ with a 256-token budget.

D. Cryptographic Provenance and Artifact Integrity

Every rollout record captures exact input token IDs, generated BPE token IDs, wall-clock inference duration, and model weight digest in an append-only event ledger. The canonical dataset 10_COMPLETE_QWEN_PROTOCOL2_ROLLOUTS.jsonl is frozen under SHA-256 digest b8792f8df375fccdbd800d440cc884ef2db2f860ff3533af01c70161a5cc1d15.

V. Statistical Methodology and Results

A. Cluster-Robust Factorial Outcomes

Table II summarizes completion accuracies across all four experimental cells under heuristic regex and structural semantic verification.

TABLE II
Protocol 2.0 Factorial Outcomes ($N = 400$ Rollouts, $C = 20$ Clusters)

State Condition	Base (π_0)	Instruct (π)	Difference (Δ)
Recovery States (s_R)	0.0000	0.0000	+0.0000
Control States (Heuristic)	0.9700	1.0000	+0.0300
Control States (Structural)	1.0000	1.0000	+0.0000
Matched Contrast (Heuristic)	–	–	−0.0300
Cluster SE / p-value	–	–	$SE = 0.0164, p = 0.0828$
95% Confidence Interval	–	–	$[-0.0643, +0.0043]$
Matched Contrast (Structural)	–	–	+0.0000 (Sample Null)
Cluster SE / p-value	–	–	Degenerate ($s_D = 0.0000$)

B. Degenerate Variance Analysis

Under structural scoring, every problem cluster $j \in \{1, \ldots, 20\}$ yields:

$$\Delta_{j,\text{Base}} = -1.0000, \quad \Delta_{j,\text{Instruct}} = -1.0000 \implies D_j = 0.0000$$

Consequently, the sample cluster standard deviation is $s_D = 0.0000$, and the cluster-robust standard error is $\text{SE} = 0.0000$. Under this condition:

- The Student-t statistic $t = \bar{D}/\text{SE} = 0/0$ is mathematically undefined.
- The conventional two-sided p-value is not computable.
- The conventional Studentized 95% confidence interval $[0, 0]$ is degenerate.

We emphasize that zero sample variance across 20 items is an empirical consequence of boundary saturation (100% control vs. 0% recovery), rather than proof of universal population equivalence.

C. Verifier Forensics and Heuristic Regex Failures

In heuristic scoring, Base Control scored 97.0% due to 3 apparent failures under seed 405 (problems fresh_004, fresh_006, fresh_013). Trace-by-trace inspection revealed that the base model correctly calculated the revenue but concluded with an unprompted summary review ending in list bullets (e.g., "- In step 3..."). The fallback regex matched the trailing bullet index 3 instead of the revenue answer. Under structural equivalence, all three rollouts are mathematically correct, establishing 100/100 (100.0%) accuracy for both checkpoints.

D. Behavioral Trace Decomposition: Continuation Inertia

We decompose all 200 recovery rollouts across five explicit process dimensions:

- Final Accuracy (Y_{acc}): Exactly 0 / 200 (0.0%) reached the gold terminal answer.
- Explicit Error Detection (Y_{det}): Exactly 0 / 200 (0.0%) emitted backtracking cues (*wait, actually, mistake, wrong*).
- Intermediate Correction (Y_{corr}): Exactly 0 / 200 (0.0%) corrected the corrupted Step 1 product.

- Faithful Error Propagation (Y_{prop}): 197 / 200 (98.50%) adopted the corrupted value E and correctly executed the downstream multiplication $E \times 4$.
- Attention-Slip Divergence (Y_{div}): Exactly 3 / 200 (1.50%) confused the unit price with the hourly rate.

E. Finite-Sample Exact Confidence Bounds

Because asymptotic inference collapses at zero events ($k = 0$), we compute exact Clopper-Pearson binomial confidence bounds [12]:

$$p_U = 1 - (\alpha/2)^{1/n}, \quad p_{U,\text{one-sided}} = 1 - \alpha^{1/n}$$

As reported in Table III, assuming 200 independent trials yields a two-sided 95% interval of $[0.000\%, 1.828\%]$ (one-sided upper bound: 1.487%). However, because rollouts within problem clusters share prefixes, treating the 20 problem clusters as the sampling unit ($C = 20$) provides a conservative cluster-level sensitivity bound of $\leq 13.911\%$ (two-sided interval: $[0.000\%, 16.843\%]$).

TABLE III
Clopper-Pearson Exact Confidence Bounds for Zero Observed Recovery ($k = 0$)

Aggregation Tier	Effective N	One-Sided 95% Upper	Two-Sided 95% CI
Rollout Level (Total)	200	1.49%	[0.00%, 1.83%]
Model Level (Per Checkpoint)	100	2.95%	[0.00%, 3.62%]
Cluster Level (Sensitivity)	20	13.91%	[**0.00%**, **16.84%**]

VI. Discussion: Mechanistic Nature of Continuation Inertia

Why do small language models exhibit near-total continuation inertia? Autoregressive language models are trained via next-token prediction to model the joint distribution $P(x_t \mid x_{<t})$. When presented with a prompt prefix containing an arithmetic error, the attention mechanism treats all prefix tokens as grounded context. In standard pretraining and instruction tuning, the model is penalized for contradicting prompt premises. Consequently, the model's most probable trajectory is to execute subsequent derivations that are logically consistent with the preceding tokens, even when those tokens contain an objective arithmetic flaw.

This demonstrates that without explicit verification prompting or external feedback tools (such as CRITIC or Reflexion), autoregressive models function as faithful continuation engines rather than autonomous error detectors.

VII. Limitations

- Model Scale: Evaluated on two 1.5B checkpoints; larger models (7B–72B) or reasoning models with internal test-time search (e.g., DeepSeek-R1) may exhibit different behaviors.
- Task Domain: Restricted to 20 elementary arithmetic rate-multiplication problems; does not evaluate open-domain theorem proving or multi-branch logic.
- Decoding Scaffolding: Evaluated under unprompted continuation without reflective scaffolding (*"check your work"*) or tool interaction.
- Observational Models: Checkpoint comparisons remain observational due to chat template differences.
- Statistical Power: With $C = 20$ problem clusters, the study has low power to detect subtle recovery rates ($|D| < 0.10$).
- Human Validation: Internal blinded human audit of 40 rollouts showed 100% concordance with structural parsing, but multi-rater crowdsourced adjudication remains pending future work.

VIII. Reproducibility & Open Source

All experimental materials, code, and neural rollout logs are permanently open-sourced at https://github.com/shamddd/recovery_eval. To reproduce all statistical tables in under 1 second:

```
git clone https://github.com/shamddd/recovery_eval.git
cd recovery_eval
python3 reproduce_results.py
python3 -m pytest tests/ -v
```

IX. Conclusion

We introduced RecoveryEval, an open-source evaluation framework for language model reasoning trajectories. By eliminating control answer leakage, 64-token truncation censoring, and parser artifacts, we demonstrated that the historical negative interaction was an evaluation artifact. Small language models exhibit near-total continuation inertia (98.5%), faithfully propagating injected errors rather than self-correcting. Our findings highlight the necessity of state-matched, provenance-aware infrastructure for reasoning diagnostics.

Declarations & AI Assistance Disclosure

In the preparation of this manuscript, the author utilized automated AI research tools (including Google Antigravity and Large Language Models) to assist with automated test suite refactoring, simulated adversarial peer review critiques, and LaTeX formatting. All primary neural rollouts were generated on independent GPU hardware (RunPod RTX 4090) using official checkpoints, and all statistical counts, exact bounds, and interpretations were verified against raw logs. The human author takes full scientific responsibility for the work. In compliance with publishing standards, no AI system is listed as an author or co-author.

References

[1] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," in arXiv preprint arXiv:2110.14168, 2021.

[2] A. Yang, B. Zhang, B. Zheng, D. Liu, J. Zhou et al., "Qwen2.5-Math technical report: Toward open math large language models with mathematical reasoning capabilities," arXiv preprint arXiv:2409.12122, 2024.

[3] H. Lightman, V. Kosaraju, Y. Shen, G. Hase, P. Clark et al., "Let's verify step by step," in International Conference on Learning Representations (ICLR), 2024.
[4] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman, "STaR: Bootstrapping reasoning with reasoning," in Advances in Neural Information Processing Systems (NeurIPS), 2022.
[5] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in Advances in Neural Information Processing Systems (NeurIPS), 2023.
[6] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, M. Huang, N. Duan, and W. Chen, "CRITIC: Large language models can self-correct with tool-interactive critiquing," in International Conference on Learning Representations (ICLR), 2024.
[7] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," in International Conference on Learning Representations (ICLR), 2024.
[8] G. Tyen, H. Mansoor, V. Cărbune, P. Chen, and T. Mak, "LLMs cannot find reasoning errors, but can correct them given the error location," in Findings of the Association for Computational Linguistics (ACL), 2024, pp. 15 049–15 065.
[9] C. Zheng, Z. Zhang, B. Zhang, D. Liu, and M. Huang, "ProcessBench: Identifying process errors in mathematical reasoning," in Findings of the Association for Computational Linguistics: EMNLP 2024, 2024.
[10] K. Stechly, M. Valenti, K. Valmeekam, and S. Kambhampati, "On the self-verification limitations of large language models on reasoning and planning tasks," arXiv preprint arXiv:2402.08115, 2024.
[11] P. Wang, L. Li, Z. Shao, R. X. Xu, D. Dai, Y. Li, D. Chen, Y. Wu, and Z. Sui, "Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations," in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), 2024, pp. 9426–9439.
[12] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," Biometrika, vol. 26, no. 4, pp. 404–413, 1934.